

ArcLab Engine

Full Custom Physics Simulation Engine Master Plan

Purpose: A practical roadmap for building a custom engine for virtual physics experiments, scientific visualization, simulation data, and future AI-driven research automation.

Prepared for: solo-founder development on Ryzen 9 9900X + RTX 2060 desktop, with i7 14th-gen + RTX 4060 laptop as secondary compute/test machine.

Date: 2026-06-04

1. Executive Overview

ArcLab Engine is a custom scientific simulation engine and editor. The aim is not to build a game engine with physics attached. The aim is to build a physics-first experimentation environment where a user can construct a virtual experiment, run it, inspect the forces and fields involved, record the results, compare runs, and eventually let an AI assistant run parameter sweeps automatically.

The first version should be narrow but impressive: a 3D magnetic particle experiment where charged particles move through electric and magnetic fields, field lines are visible, real-time graphs update while the simulation runs, and users can save and reload the experiment. This demo is the seed of the whole platform because it proves rendering, physics, UI, data recording, graphing, experiment control, and scientific visualization in one integrated product.

Going full custom is the hard path. It means you build the window, renderer, editor, math layer, simulation loop, physics solvers, data recording, memory management, and eventually GPU compute yourself. The advantage is control. The disadvantage is time. You will move slower at the beginning than someone using Unity, Unreal, or Godot, but you will build a platform that can become specialized for scientific experiments instead of being trapped inside a game-engine design philosophy.

Core decision

Build the engine around experiments, not around levels, characters, and gameplay. Every subsystem should answer: does this help a user create, run, inspect, reproduce, or automate a physics experiment?

1.1 What this project is

- A real-time 3D experiment editor.
- A simulation engine with custom physics modules.
- A scientific visualization platform for particles, fields, forces, heat, and motion.
- A data recording and analysis tool that exports useful experiment results.
- A future AI automation layer that can generate experiments, run sweeps, compare outputs, and suggest next tests.

1.2 What this project is not, at the start

- It is not a full quantum mechanics platform on day one.
- It is not a complete chemistry engine on day one.
- It is not a nuclear-element discovery system on day one.
- It is not a multiplayer cloud product on day one.
- It is not a replacement for Ansys, COMSOL, CERN software, or national-lab simulation stacks at MVP stage.

1.3 The correct first demo

The first public demo should be called Magnetic Particle Confinement. It should show two electromagnetic coils, a charged particle emitter, visible magnetic field lines, blue/red particle trails, live graphs, and a control panel. The user should be able to change particle speed, charge, mass, field

strength, emitter angle, timestep, and simulation speed. The engine should display kinetic energy, particle count, average velocity, and confinement time.

This demo is better than a generic bouncing-ball demo because it communicates the unique identity of the project. A bouncing-ball demo looks like every physics tutorial. A charged-particle experiment with field lines looks like ArcLab.

2. Product Vision and Design Philosophy

2.1 The product vision

ArcLab should feel like a mixture of a 3D engine editor, a physics laboratory, an oscilloscope, and a research notebook. The user should not only see a cool simulation. The user should understand why the simulation behaves that way. This means the interface must expose variables, equations, force vectors, field directions, sensor readings, and experiment history.

The long-term vision is a virtual physics lab where a scientist, student, engineer, or company can test ideas before spending money on real hardware. A company could test thermal flow. A teacher could demonstrate electromagnetism. A researcher could run a fast approximation before moving to expensive high-fidelity software. Later, AI could explore large design spaces automatically.

2.2 Engine philosophy

Principle	Meaning	Design consequence
Experiment-first	The core object is an experiment, not a game level.	Every scene has parameters, sensors, graphs, results, and reproducibility metadata.
Scientific visibility	Invisible forces must become visible.	Render vectors, field lines, scalar maps, trajectories, collision points, and energy changes.
Reproducibility	The same input should produce the same result when deterministic mode is enabled.	Fixed timesteps, seeded randomness, versioned settings, and recorded metadata.
Accuracy tiers	Users need speed sometimes and accuracy other times.	Provide preview mode, accurate mode, and research mode later.
Data is first-class	A simulation without data is only animation.	Record CSV/JSON, live graphs, sensors, snapshots, and result summaries.
Local-first	Your first product should run locally on your desktop.	No cloud dependency in the first year.

2.3 MVP promise

The MVP promise should be simple: build physics experiments visually, run them, see what happens, and export the data. That is enough. Do not try to sell it as a professional replacement for established simulation tools yet. The first target is proof of capability and visual clarity.

3. Hardware Reality: Your Machines

3.1 Available machines

Machine	CPU	GPU	Recommended role
Desktop	Ryzen 9 9900X	RTX 2060, likely 6 GB VRAM	Main development machine, engine builds, CPU simulation, long-running tests, storage-heavy project work.
Laptop	Intel i7 14th gen	RTX 4060 laptop, commonly 8 GB VRAM	Secondary testing, CUDA experiments, shader experiments, AI model tests, portable work.

Use the desktop as the main workstation because airflow, sustained CPU performance, storage expansion, and long-running stability matter more than peak laptop GPU speed. Use the laptop to compare performance and test newer GPU features. The laptop GPU may be faster than the desktop RTX 2060, but laptop cooling and power limits can reduce sustained performance.

3.2 What your hardware can handle

Workload	Desktop expectation	Laptop expectation	Notes
Engine coding and compilation	Strong	Good	Ryzen 9 desktop is ideal for long compile/test cycles.
3D editor viewport	Good	Good to strong	RTX 2060 is enough for early viewport rendering.
10k-100k CPU particles	Good	Good	Depends heavily on algorithm and data layout.
1M particles CPU-side	Possible but expensive	Possible but expensive	Needs optimized memory layout and broadphase where relevant.
1M particles GPU-side	Possible later	Better later	Requires compute shaders/CUDA and careful GPU buffer management.
Local AI assistant	Limited on 2060	Better on 4060	VRAM is the constraint; use small quantized models.
Quantum/nuclear serious simulation	Not realistic at research scale	Not realistic at research scale	Can build educational/prototype modules, not national-lab-scale models.

4. Recommended Technology Stack

4.1 Core stack

Layer	Choice	Why
Language	C++20 or C++23	Best balance of performance, ecosystem, low-level control, and engine tradition.

Build system	CMake	Standard for C++ engine projects and third-party libraries.
Window/input	SDL3 or GLFW	Simple cross-platform windowing and input. SDL3 is broader; GLFW is minimal.
Graphics API	Vulkan long-term; OpenGL possible for prototype	Vulkan gives explicit control and compute path. OpenGL is faster to start.
UI	Dear ImGui	Fastest way to build an internal editor UI.
Math	GLM first, custom math later	Avoid wasting early months rewriting vector/matrix basics.
Serialization	JSON first, binary later	JSON is readable and debuggable for early experiments.
Plotting	ImPlot	Works naturally with Dear ImGui for live graphs.
GPU compute later	Vulkan compute or CUDA	Vulkan compute is integrated; CUDA is productive but NVIDIA-only.

4.2 OpenGL first vs Vulkan first

There are two valid routes. The practical route is OpenGL first, Vulkan later. The serious-engine route is Vulkan first. Vulkan first is painful but makes you learn the correct architecture earlier. OpenGL first gives you a faster first demo. If your motivation depends on seeing progress quickly, use OpenGL for the first 90 days and design the renderer abstraction so Vulkan can replace it later. If you are prepared for slow setup and debugging, start with Vulkan.

Route	Pros	Cons	Recommendation
OpenGL first	Fast start, simpler tutorials, quicker visual demo.	Legacy API, less explicit control, compute path less future-proof.	Best if you need motivation and a demo quickly.
Vulkan first	Modern, explicit, good for compute and serious renderer design.	Steep learning curve; first triangle can take a long time.	Best if you accept slow early progress and want long-term control.

Recommendation

For this project, I would do OpenGL for the first visual MVP only if you feel Vulkan will slow you down too much. Otherwise, use Vulkan from day one and accept that the first month will feel brutal. The engine is long-term, so Vulkan is justified.

5. High-Level Architecture

ArcLab should be built as a core engine plus an editor application. The engine should not depend on the editor. The editor uses the engine. This separation matters because later you may want command-line simulations, batch runs, server-side simulation jobs, or AI agents that run experiments without opening the full UI.

5.1 Main layers

ArcLab/
engine/
 core/ logging, time, application, jobs, memory
 platform/ windows, input, files, OS utilities
 math/ vectors, matrices, quaternions, transforms
 renderer/ graphics API abstraction, meshes, shaders, debug draw
 physics/ rigid bodies, particles, collisions, fields, integrators
 simulation/ experiment runtime, timestep, sensors, recorders
 editor_ui/ reusable editor widgets and panels
 serialization/ JSON, binary snapshots, asset manifests
 assets/ mesh, shader, material, texture loading
 tools/ profilers, inspectors, import/export helpers
 editor/ ArcLab Editor application
 sandbox/ small test applications
 experiments/ saved demo experiments
 third_party/ external libraries
 docs/ architecture docs, notes, formulas, planning

5.2 Runtime loop

```
while app.running:
    input.poll()
    editor.process_ui_input()

    if simulation.playing:
        accumulator += frame_dt
        while accumulator >= fixed_dt:
            simulation.step(fixed_dt)
            recorder.sample_if_needed()
            accumulator -= fixed_dt

    renderer.begin_frame()
    renderer.render_scene(scene)
    renderer.render_debug_vectors()
    editor.render_panels()
    renderer.end_frame()
```

5.3 Subsystem responsibilities

Subsystem	Owns	Should not own
Core	Application lifecycle, logging, time, memory, job system.	Physics equations or rendering logic.
Renderer	GPU resources, draw commands, shaders, viewport.	Simulation truth or experiment results.
Physics	Motion, forces, particles, fields, collisions, constraints.	UI panels or file dialogs.
Simulation	Experiment state, fixed timestep, sensors, recorders, play/pause/reset.	Low-level GPU API details.
Editor	Scene explorer, property panel, graph panel, controls, selection.	Core physics algorithms.

Serialization	Saving/loading scenes, snapshots, experiment configs.	Making decisions about simulation behavior.
---------------	---	---

6. Engine Data Model

6.1 Entity-component model

Use a simple entity-component model. Do not build a massive ECS framework at the start. A clean component registry with arrays is enough. The aim is data clarity and fast iteration, not architecture cosplay.

```
EntityID entity = scene.create_entity("Particle Emitter");
scene.add<TransformComponent>(entity, Transform{...});
scene.add<ParticleEmitterComponent>(entity, ParticleEmitter{...});
scene.add<FieldSensorComponent>(entity, FieldSensor{...});
```

6.2 Core components

Component	Fields	Purpose
TransformComponent	position, rotation, scale	Every visible or simulated object needs a transform.
MeshComponent	mesh handle, material handle	Makes an entity renderable.
RigidBodyComponent	mass, velocity, angular velocity, forces	Classical physics objects.
ColliderComponent	shape type, bounds, material	Collision detection and response.
ParticleEmitterComponent	rate, lifetime, initial velocity, charge distribution	Creates particles for experiments.
FieldComponent	field type, strength, direction, falloff	Electric, magnetic, gravity, or custom vector field.
SensorComponent	sample rate, measured quantity, target filter	Records data from the simulation.
GraphComponent	source sensor, axes, display settings	Turns recorded data into live plots.

6.3 Experiment object

The experiment should be the top-level project file. It should store the scene, simulation settings, recording settings, random seed, units, engine version, and results. This gives you reproducibility later.

```
Experiment
  name
  description
  engine_version
  unit_system
  random_seed
  simulation_settings
    fixed_dt
    solver_iterations
  accuracy_mode
  max_particles
```

- scene
 - entities
 - components
- recorders
 - sensors
 - sample_rates
 - export_settings
- results
 - optional cached summaries

7. Math and Numerical Foundation

7.1 What math you need first

- Vectors and matrices
- Dot product and cross product
- Quaternions
- Transforms
- Rays and intersections
- AABB and bounding spheres
- Basic calculus ideas
- Numerical integration
- Floating point precision limits

7.2 Fixed timestep

Use a fixed timestep for physics. Rendering can run at variable FPS, but physics should step at a stable interval such as 1/60 or 1/120 second. This prevents physics from changing when frame rate changes.

```
const double fixed_dt = 1.0 / 120.0;
accumulator += frame_dt;
while (accumulator >= fixed_dt) {
    physics_step(fixed_dt);
    accumulator -= fixed_dt;
}
```

7.3 Integrators

Integrator	Pros	Cons	Use
Explicit Euler	Very simple.	Unstable for many systems.	Only for learning and quick tests.
Semi-implicit Euler	Simple and more stable.	Still not high-accuracy.	Good default for early rigid bodies and particles.
Velocity Verlet	Good energy behavior for particles.	More state and complexity.	Useful for molecular/particle systems later.
Runge-Kutta 4	High accuracy for smooth ODEs.	More expensive.	Useful for field line tracing and analysis tools.

Start with semi-implicit Euler. Add RK4 for field line tracing. Add better integrators only when the simulation demands it.

8. Renderer Plan

8.1 Renderer MVP

The renderer should first support a grid, camera, basic meshes, debug lines, particles, and simple materials. Do not chase photorealism. Scientific clarity is more important than cinematic graphics. The renderer should make forces, paths, and field structures legible.

Feature	Priority	Reason
Grid floor	Immediate	Gives scale and orientation.
Perspective camera	Immediate	Required for 3D navigation.
Debug lines	Immediate	Needed for vectors, rays, field lines, axes.
Instanced particles	High	Needed for particle simulations.
Trail rendering	High	Critical for visualizing trajectories.
Text labels	Medium	Useful for axes and annotations.
Volumetric effects	Later	Nice for heat/plasma but not MVP.
PBR materials	Later	Looks good but not essential for scientific MVP.

8.2 Debug visualization is not optional

In a custom physics engine, debug visualization is survival. You need to see vectors, bounding boxes, collision normals, contact points, field directions, and sensor regions. Without debug draw, you will waste days guessing why physics is wrong.

Debug visual	Shows
Force arrow	Direction and magnitude of force on an object.
Velocity arrow	Current direction and speed.
Collision normal	Direction of collision response.
AABB box	Broadphase collision bounds.
Field lines	Shape of electric/magnetic/gravity field.
Sensor volume	Where data is being sampled.
Trajectory trail	Path through time.

9. Physics Plan

9.1 First physics module: classical mechanics

Start with classical mechanics. This means mass, force, velocity, acceleration, gravity, drag, friction, and collisions. The reason is simple: every other module depends on reliable time integration and scene state management.

```
force_sum = gravity_force + external_forces + drag_force
acceleration = force_sum / mass
velocity += acceleration * dt
```

```
position += velocity * dt
```

9.2 Collision system

Do collisions in stages. First, sphere-plane and sphere-sphere. Then AABB. Then broadphase spatial grids. Do not jump straight into complicated convex collision algorithms. The first target is stable and debuggable behavior.

Stage	Collision types	Purpose
Stage 1	Sphere-plane, sphere-sphere	Make objects bounce and interact.
Stage 2	AABB-plane, AABB-AABB	Support boxes and simple editor objects.
Stage 3	Spatial grid broadphase	Avoid checking every object against every other object.
Stage 4	OBB and capsule	Better shapes for tools and demos.
Stage 5	Convex collision	Advanced only after basics are strong.

9.3 Particle physics

Particles are the visual identity of ArcLab. They let you show fields, trails, flows, and energy changes. Implement particles as a separate optimized system, not as thousands of heavy entities. Each particle should be simple and stored in contiguous arrays for cache efficiency.

```
struct Particle {  
    Vec3 position;  
    Vec3 velocity;  
    Vec3 force;  
    float mass;  
    float charge;  
    float temperature;  
    float lifetime;  
    uint32_t flags;  
};
```

9.4 Electromagnetism MVP

Electromagnetism should be the first signature scientific module. You do not need a full Maxwell solver at the start. Begin with simplified electric and magnetic fields that apply force to charged particles. Use the Lorentz force equation as the basis.

$$F = q * (E + \text{cross}(v, B))$$

where:

q = particle charge

E = electric field vector

v = particle velocity

B = magnetic field vector

This lets you build a real demo where particles curve, spiral, accelerate, or get confined depending on field settings. It is visual, understandable, and technically meaningful.

9.5 Heat module later

Heat simulation should come after particles and fields. Start with simple heat diffusion on a grid or between connected objects. Do not start with full fluid convection. A simple heat-map over a plane or object is enough for the first thermal demo.

9.6 Fluid module later

Fluids are difficult and can consume huge time. Delay serious fluids until the engine has stable rendering, particles, graphs, save/load, and profiling. For early visuals, use particle-flow approximations. Later explore grid-based fluid solvers, SPH, or simplified smoke simulation.

10. Memory and Storage Constraints

10.1 Why memory will become a major issue

The engine itself will not be huge. The problem is simulation data. If you store every particle, every frame, every field value, every sensor reading, and every graph sample without limits, your memory and disk usage will explode. Scientific tools must record data, but they must record intelligently.

10.2 Particle memory estimate

Particle count	Approx memory at 64 bytes each	Practical meaning
10,000	0.64 MB	Easy. Good for first demos.
100,000	6.4 MB	Still easy on CPU/GPU.
1,000,000	64 MB	Possible, but update/render strategy matters.
10,000,000	640 MB	Heavy. Requires GPU compute and careful buffers.
50,000,000	3.2 GB	Research/benchmark territory, not early MVP.

10.3 History memory problem

Rewind and time history are dangerous. One million particles may be around 64 MB for one state. If you store 600 frames, that becomes about 38.4 GB. That is only particle state, not graphs, fields, logs, or rendering resources.

1,000,000 particles * 64 bytes = 64 MB per frame
64 MB * 600 saved frames = 38.4 GB

10.4 Memory rules

- Do not store all particle data for every frame by default.
- Use ring buffers for short-term rewind.
- Use sparse snapshots for long-term history.
- Let users choose what sensors record.

- Record aggregate data unless full trajectory data is needed.
- Compress old results when saving to disk.
- Keep simulation state separate from render state.
- Track RAM and VRAM usage in the editor status bar from early development.

10.5 Recommended memory design

Data type	Storage strategy
Current simulation state	Hot memory, contiguous arrays, optimized for update speed.
Short rewind history	Ring buffer storing snapshots every N frames.
Sensor data	Time-series arrays with fixed sample rates.
Graphs	Downsampled display cache separate from raw data.
Long-term results	Compressed files on disk.
GPU particle buffers	Double-buffered or persistently mapped buffers later.

10.6 Disk space expectations

Project maturity	Estimated storage	What uses the space
Early engine	20-50 GB	Code, dependencies, build files, shaders, sample assets.
Serious prototype	100-250 GB	Build variants, recorded demos, experiment outputs, logs.
AI + simulation prototype	250-750 GB	Local models, datasets, cached results, many recordings.
Research-heavy phase	1-5 TB+	Molecular data, high-resolution particle outputs, batch experiments.

Storage recommendation

Use at least 1 TB free space for comfortable development. A 2 TB NVMe SSD is the better target. Keep raw simulation outputs on a separate drive once the project grows.

11. Performance Architecture

11.1 The biggest performance mistake

The biggest mistake is checking every particle or object against every other particle or object. This creates $O(n^2)$ behavior. Ten thousand objects can become 100 million checks. One million objects is impossible with naive pairwise logic. You need spatial organization.

11.2 Broadphase approaches

Method	Best for	Difficulty
Uniform grid	Particles and many similar objects.	Low to medium. Start here.
Spatial hash	Large sparse worlds.	Medium. Good later.
Quadtree/octree	Hierarchical space partitioning.	Medium to high. Useful for uneven distributions.
BVH	Meshes and ray queries.	High. Use later for advanced collisions/rendering.

11.3 Data-oriented design

For particles and physics, prefer arrays of data over thousands of heap-allocated objects. CPU cache matters. A clean data layout can make a bigger difference than clever equations.

```
// Better for particle loops
positions[]
velocities[]
charges[]
masses[]

// Worse for particle loops
ParticleObject* particles[] where every particle is a separate heap object
```

11.4 Profiling from early development

Add performance counters early. If the engine cannot tell you where time goes, optimization becomes guessing.

Metric	Why it matters
Frame time	More useful than FPS for optimization.
Simulation time	Shows whether physics is the bottleneck.
Render time	Shows whether GPU/draw calls are the bottleneck.
UI time	ImGui panels can become expensive if careless.
Particle count	Core stress metric.
Collision checks	Detects broadphase problems.
RAM used	Prevents silent memory growth.
VRAM used	Critical on RTX 2060/4060.
Disk output per run	Prevents experiment data from exploding.

12. Editor Plan

12.1 Required panels

Panel	Purpose
Scene Explorer	List all objects, fields, sensors, emitters, and graphs.
Properties	Edit selected object parameters.
Viewport	3D experiment view.
Simulation Controls	Play, pause, reset, timestep, accuracy mode.
Data Monitors	Live values from sensors.
Graph Panel	Live plots of selected variables.
Console	Engine messages, warnings, commands.
Profiler	Frame time, memory, simulation stats.
Asset/Experiment Browser	Load saved experiments and reusable components.

12.2 User workflow

```
Create experiment
-> Add field object
-> Add particle emitter
-> Add sensor
```

- > Press Play
- > Watch particles and field lines
- > Change variables
- > Compare graphs
- > Export CSV
- > Save experiment

12.3 UI design rule

The UI should always make cause and effect visible. If the user changes field strength, the field lines, particle path, and graphs should make the effect obvious. The best demo is not just beautiful; it teaches the viewer what happened.

13. Save/Load, Data, and Experiment Reproducibility

13.1 Save format

Use JSON for early saves because you can read it, debug it, and manually edit it. Use a binary format later for large snapshots. Separate experiment settings from result data so a user can share a small experiment file without gigabytes of recorded output.

```
{
  "name": "Magnetic Particle Confinement Demo",
  "engine_version": "0.1.0",
  "random_seed": 12345,
  "simulation": {
    "fixed_dt": 0.008333,
    "accuracy": "preview",
    "max_particles": 100000
  },
  "entities": [
    { "id": 1, "name": "Emitter", "components": {...} },
    { "id": 2, "name": "Magnetic Field", "components": {...} }
  ]
}
```

13.2 Export formats

Format	Use
CSV	Simple graph and sensor exports.
JSON	Experiment configuration and small structured results.
Binary snapshot	Large state saves and fast reloads later.
HDF5 later	Scientific datasets when the project becomes serious.
PNG/MP4 later	Visual exports and demos.

13.3 Metadata to record

- Engine version
- Simulation module versions
- Timestep

- Accuracy mode
- Solver iterations
- Random seed
- Hardware used
- Number of particles/objects
- Units
- Date/time
- Parameter values
- Export settings

14. AI Assistant Roadmap

14.1 AI is not phase one

Do not build AI before the simulator works. AI needs tools to control. First build the engine, editor, sensors, graphs, save/load, and batch-run capability. Then AI can operate those systems.

14.2 What AI should do first

AI capability	Description
Explain simulation	Read current settings and explain what variables mean.
Suggest experiments	Recommend changes to test a hypothesis.
Parameter sweep	Run many experiments with different values.
Compare outputs	Rank runs by a metric such as confinement time.
Generate report	Summarize experiment setup, results, graphs, and next steps.

14.3 Tool interface for AI

Design internal commands before adding the AI model. The AI should call safe engine commands, not directly mutate memory. This keeps the system debuggable and safer.

```

create_experiment(name)
add_particle_emitter(params)
add_magnetic_field(params)
set_parameter(entity, property, value)
run_simulation(duration)
get_sensor_data(sensor_id)
export_results(path)
compare_runs(metric)

```

15. Roadmap

15.1 First 30 days

Days	Goal	Deliverable
1-5	Project setup	CMake, repo structure, third_party folder, logging.

6-10	Window and loop	Window opens, input works, FPS shown.
11-15	Renderer start	Clear screen, draw triangle/cube/grid.
16-20	Camera	WASD/mouse camera or orbit camera.
21-25	ImGui editor shell	Docking layout with panels.
26-30	Scene objects	Spawn cube/sphere, transform editing.

15.2 Days 31-60

Days	Goal	Deliverable
31-38	Basic physics	Gravity, velocity, force, mass.
39-45	Collision basics	Sphere-plane and sphere-sphere.
46-52	Particle emitter	Particles spawn and move.
53-60	Particle rendering	Instanced particles and trails.

15.3 Days 61-90

Days	Goal	Deliverable
61-68	Electric/magnetic fields	Apply Lorentz force to charged particles.
69-75	Field visualization	Field lines and vector arrows.
76-82	Graphs and sensors	Live kinetic energy/velocity/particle count graphs.
83-90	Save/export/polish	Save/load experiment and export CSV.

15.4 Month 4-6

- Improve renderer abstraction.
- Add spatial grid broadphase.
- Add better data recording.
- Add experiment browser.
- Add thermal prototype.
- Add batch-run mode.
- Create website and demo video.
- Publish first devlog.

15.5 Month 7-12

- GPU particle compute prototype.
- More accurate field solvers.
- AI assistant prototype.
- Parameter sweeps.
- Comparison dashboard.
- Public alpha build.
- Documentation and tutorials.

- First educational/engineering use cases.

16. Detailed Milestone Deliverables

Milestone	Definition of done
M0 - Engine boots	App opens reliably, logs startup, handles input, closes cleanly.
M1 - 3D viewport	Camera, grid, basic mesh rendering, object transforms.
M2 - Editor shell	Dockable panels, scene explorer, properties, console.
M3 - Physics basics	Objects fall, move, bounce, and reset deterministically.
M4 - Particles	Emitter spawns at least 100k particles with trails.
M5 - Fields	Electric/magnetic field changes charged particle paths.
M6 - Data	Sensors record values; live graphs update; CSV export works.
M7 - Save/load	Experiment can be saved, reopened, and replayed.
M8 - Demo polish	Magnetic confinement demo looks impressive and understandable.
M9 - Batch mode	Run experiment with multiple parameter values automatically.
M10 - AI control	AI triggers parameter sweeps through safe engine commands.

17. Risk Register

Risk	Impact	Prevention
Vulkan complexity slows progress	High	Use a minimal renderer first; keep OpenGL fallback option.
Scope creep into quantum/nuclear too early	High	Lock first demo to magnetic particles and data recording.
Memory explosion from history recording	High	Use ring buffers, sensor sampling, and no unlimited frame history.
Physics instability	High	Fixed timestep, stable integrators, debug draw, deterministic tests.
Performance bottlenecks	Medium/high	Profile early; use contiguous arrays; avoid $O(n^2)$ logic.
UI becomes messy	Medium	Use clear panels and keep first workflow simple.
No public demo for too long	High	Make the magnetic demo the 90-day target.
Trying to build perfect architecture	Medium	Keep systems replaceable but simple. Ship working milestones.

18. What Not To Build Yet

- Full quantum mechanics
- New element synthesis simulation
- Full chemistry engine
- Full fluid engine
- Cloud collaboration

- Mobile app
- Multiplayer
- Plugin marketplace
- Photoreal rendering
- Custom programming language
- Massive ECS framework
- Perfect UI framework
- Real lab hardware integration

These ideas may come later, but adding them too early will destroy momentum. The first win is a working custom simulation engine with a beautiful and understandable physics demo.

19. Learning Plan

19.1 Learn in this order

Order	Topic	Why
1	C++ fundamentals	Engine code lives here.
2	CMake and project structure	You need repeatable builds.
3	Linear algebra	Everything in 3D depends on it.
4	Rendering basics	You need to see the simulation.
5	Dear ImGui	Fast editor creation.
6	Physics integration	Motion and forces.
7	Collision detection	Object interaction.
8	Data-oriented design	Particle performance.
9	Profiling	Optimization discipline.
10	GPU compute	Large simulations later.
11	Numerical methods	Accuracy and stability.
12	Scientific visualization	Make invisible physics visible.

19.2 Practice projects

- Render a grid and movable camera.
- Create a transform gizmo.
- Drop 100 spheres onto a plane.
- Render 100k particles.
- Draw vector arrows for velocity.
- Make charged particles curve in a magnetic field.
- Save an experiment to JSON and reload it.
- Export velocity data to CSV and graph it.

20. Final Recommended Build Order

1. Window + logging + input
2. Renderer + grid + camera
3. ImGui editor shell
4. Entity/component system

- 5. Transform editing
- 6. Basic physics integration
- 7. Collision basics
- 8. Particle system
- 9. Electric/magnetic field module
- 10. Field-line visualization
- 11. Sensors and graphs
- 12. Save/load
- 13. CSV export
- 14. Magnetic confinement demo
- 15. Profiler and memory tracker
- 16. Spatial grid optimization
- 17. GPU particle compute
- 18. Batch experiment runner
- 19. AI assistant tool interface
- 20. Advanced physics modules

21. First Public Demo Specification

21.1 Demo name

Magnetic Particle Confinement Experiment

21.2 Visual composition

- Dark 3D lab grid.
- Two glowing electromagnetic coils.
- Particle emitter shooting charged particles.
- Blue/red field lines.
- Particles spiraling through the field.
- Live graphs at the bottom.
- Scene explorer on the left.
- Simulation controls on the right.
- AI assistant panel disabled or mocked until later.

21.3 Adjustable parameters

Parameter	Range/example
Magnetic field strength	0-20 Tesla visual scale for demo purposes.
Particle charge	Positive, negative, neutral.
Particle mass	Preset values or custom.
Emitter speed	Low to high.
Emitter angle	3D direction control.
Particle lifetime	Seconds.
Simulation timestep	1/60, 1/120, 1/240.
Trail length	Short, medium, long.

21.4 Data outputs

- Average velocity over time.

- Kinetic energy over time.
- Number of active particles.
- Average distance from center.
- Confinement time.
- Particle losses through boundary.
- CSV export.

22. Conclusion

Building a full custom engine is the hardest option, but it is aligned with the long-term vision. The correct strategy is to avoid pretending the first version will be a complete virtual universe. Instead, build a tight custom engine foundation, prove it with one strong experiment, and then grow the platform module by module.

The desktop should be the main development machine. Your hardware is enough for the first serious year if you manage memory carefully and avoid trying to simulate huge systems too early. The first major bottleneck will not be hardware. It will be engine architecture, math correctness, physics stability, debugging discipline, and scope control.

The right first mission is clear: build ArcLab Engine until it can create, run, visualize, record, save, and export a magnetic particle experiment. Once that exists, you have the foundation for everything else.

23. Useful Resources To Research

Area	Resources/keywords to search
C++ engine setup	CMake, vcpkg, SDL3, GLFW, project architecture.
Rendering	Vulkan Tutorial, LearnOpenGL, GPU instancing, debug draw.
UI	Dear ImGui, ImPlot, docking branch.
Math	3D Math Primer, quaternions, transforms, linear algebra.
Physics	Game Physics Pearls, Real-Time Collision Detection, numerical integration.
Particles	data-oriented design, struct of arrays, GPU particle systems.
Fields	Lorentz force, vector fields, field line tracing, RK4.
Scientific data	CSV, JSON, HDF5, time-series data, reproducible experiments.
GPU compute	Vulkan compute shaders, CUDA, GPU buffers.
Future scientific tools	OpenFOAM, LAMMPS, Geant4, Quantum ESPRESSO, MOOSE, AiiDA.